Eötvös Loránd University

Faculty of Informatics

Dept. of Software Technology and Methodology

FixedIncome: A Full-Stack Web Application for Bond Portfolio Management

Supervisor:
Daniel Varga
Assistant Lecturer

Author:
Erlan Akbaraliev
Computer Science BSc

Budapest, 2025

Contents

1 Introduction **4**

1.1 Motivation 4

1.2 Goals and Scope 5

1.3 Structure of the Thesis 6

2 User Documentation **7**

2.1 System Requirements 7

2.2 Installation and Startup 7

2.3 Authentication and Roles 8

2.4 Application Pages 9

2.4.1 Home — Bond Table 9

2.4.2 Bond Analytics Dashboard 10

2.4.3 Treasury Yield Dashboard 11

2.4.4 Issuer Management 11

2.4.5 Transaction Management 12

2.4.6 User Management 12

2.5 CSV Import and Export 13

2.5.1 Export 13

2.5.2 Import 13

3 Developer Documentation **16**

3.1 System Architecture 16

3.2 Technology Stack 18

3.2.1 Backend 18

3.2.2 Frontend 18

3.3 Data Model 18

3.3.1 Issuer 19

3.3.2 Bond 19

3.3.3 Transaction 20
3.3.4 TreasuryYieldObservation 20
3.3.5 UserProfile 20
3.4 REST API Design 20
3.4.1 Authentication 20
3.4.2 Endpoint Overview 21
3.4.3 Filtering, Searching, and Ordering 21
3.4.4 Pagination 21
3.4.5 Bond Analytics Endpoint 22
3.5 Permission System 23
3.6 Treasury Yield Integration 23
3.6.1 Alpha Vantage API 23
3.6.2 Sync Flow 23
3.6.3 Dashboard Computation 24
3.7 Frontend Architecture 24
3.7.1 Project Structure 24
3.7.2 Authentication Flow 25
3.7.3 Routing and Access Control 25
3.7.4 Data Tables 25
3.7.5 Charts 26
3.8 Logging 26
3.9 Containerisation 26
3.10 CI Pipelines 26
3.10.1 Code Quality Pipeline 27
3.10.2 Build and Release Pipeline 27

4 Testing **28**
4.1 Testing Strategy 28
4.2 Backend Tests 28
4.2.1 Test Framework 28
4.2.2 Running the Tests 28
4.2.3 Test Coverage Areas 29
4.3 Frontend Tests 30
4.3.1 Test Framework 30

4.3.2 Running the Tests 30
4.3.3 Test Coverage Areas 30
4.4 Code Quality 31

5 Summary **32**
5.1 Results 32
5.2 Lessons Learned 33
5.3 Future Work 33

Acknowledgements **35**

Bibliography **35**

List of Figures **38**

List of Tables **39**

List of Codes **40**

Chapter 1

Introduction

1.1 Motivation

A bond is a loan. When a government or company needs to raise money, it can borrow from thousands of investors at once by issuing bonds. Each investor lends a fixed amount (the *face value*), receives regular interest payments (the *coupon*), and gets the principal back on a fixed future date (the *maturity date*). In return for accepting a fixed repayment schedule, investors need to track several things simultaneously: which bonds they hold, who issued them, how creditworthy those issuers are, when the bonds mature, and how the interest rates on offer compare to the broader market.

Managing even a modest collection of bonds quickly becomes complicated. Consider a small investment team that holds 50 bonds from 20 different issuers across multiple countries and industries. They need to answer questions like: *What share of our portfolio matures within the next year? How exposed are we to issuers rated below investment grade? Are the coupon rates we locked in still competitive given where US Treasury yields are today?* Without dedicated tooling, the team falls back on spreadsheets — which break under concurrent edits, offer no audit trail, and require manual copy-paste every time a new bond is added or a trade is executed.

Professional solutions exist, but they come with significant barriers. Bloomberg terminals cost tens of thousands of dollars per year per seat. Generic portfolio trackers are designed for equities and lack the bond-specific fields (ISIN, coupon rate, maturity date, credit rating) needed for meaningful fixed-income analysis.

This thesis presents *FixedIncome*, a full-stack web application that addresses this gap at a practical scale. It gives a team a shared, structured catalogue of bonds and issuers, a log of buy and sell transactions, visual analytics over the portfolio composition, and a live view of US Treasury yield curves — all in one place, accessible from a browser, and deployable on any machine with Docker. The project was built as a practical exercise in modern web engineering, applying industry-standard technologies and software-engineering principles throughout.

1.2 Goals and Scope

The primary goals of the project are:

- Design and implement a relational data model that faithfully represents the domain concepts of fixed-income investing: issuers, bonds, transactions, and macroeconomic yield observations.
- Expose the data through a RESTful JSON API with JWT-based authentication and a fine-grained role system (Admin, Editor, Viewer).
- Build a responsive single-page application (SPA) that consumes the API and provides data-entry forms, filterable tables, and analytics dashboards.
- Integrate the Alpha Vantage `TREASURY_YIELD` endpoint to fetch and persist US Treasury constant-maturity yields (2-year, 10-year, 30-year), and derive the yield curve and the 10Y − 2Y spread.
- Support bulk data operations: CSV import with preview and conflict detection, streaming CSV export, and bulk deletion.
- Containerise the entire system with Docker Compose so that the application can be reproduced on any machine with a single command.
- Implement structured server-side logging so that if the application crashes or behaves unexpectedly at any time, the root cause can be identified from the log output rather than guesswork — every API request, permission denial, and validation failure is recorded with the acting user and timestamp.
- Set up Azure Pipelines for continuous integration, automating code quality checks and build verification on every pull request to the main branch.

The scope is intentionally limited to a single-user-organisation model; multi-tenancy and real-time price feeds are identified as future extensions.

1.3 Structure of the Thesis

Chapter 2 describes the application from the user's perspective, walking through each screen and feature. Chapter 3 covers the technical implementation in depth, discussing the architecture, data model, API design, frontend structure, and the external data integration. Chapter 4 presents the testing strategy and results. Chapter 5 summarises the work, reflects on lessons learned, and outlines directions for future development.

Chapter 2

User Documentation

2.1 System Requirements

FixedIncome is a web application delivered as a set of Docker containers. The only prerequisite for running the system locally is Docker Desktop (version 24 or later) with the Compose plugin. No language runtimes, databases, or package managers need to be installed separately on the host machine.

Component	Requirement
Docker Engine	24.0 or later
Docker Compose	v2 plugin
Available RAM	1 GB minimum
Available disk	2 GB (images + database volume)
Browser	Any modern browser (Chrome, Firefox, Edge, Safari)

Table 2.1: System requirements for running FixedIncome locally.

2.2 Installation and Startup

Clone the repository and navigate to the `FixedIncome` directory. Start all services with:

```
docker compose up --build
```

Code 2.1: Starting the application with Docker Compose.

Docker Compose will build the backend and frontend images, start a PostgreSQL 16 database, apply all Django migrations automatically, and serve the frontend on `http://localhost:5173` and the backend API on `http://localhost:8000`.

To stop the application:

```
docker compose down
```

Code 2.2: Stopping the application.

To stop and remove all data (including the database volume):

```
docker compose down -v
```

Code 2.3: Full teardown including database volume.

2.3 Authentication and Roles

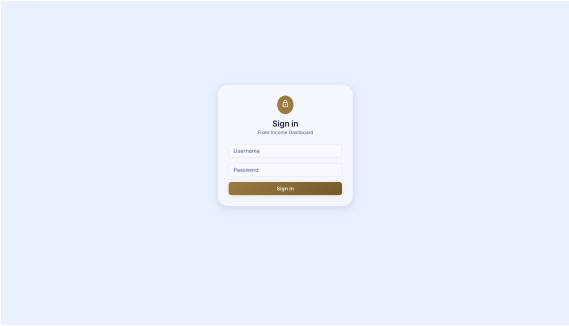


Figure 2.1: Login

Every page except the login screen is protected. Users must authenticate with a username and password. The backend issues a short-lived JWT access token (30 minutes) and a long-lived refresh token (7 days). The frontend stores these tokens and transparently refreshes the access token before it expires.

The system defines three roles, summarised in Table 2.2.

Role	Permissions
Admin	Full access: create, edit, and delete bonds, issuers, transactions, and users; trigger Treasury yield sync; bulk delete; CSV import/export.
Editor	Create and edit bonds, issuers, and transactions; CSV import/export; trigger Treasury yield sync. Cannot manage users or delete records.
Viewer	Read-only access to all data. Cannot create, edit, or delete anything.

Table 2.2: User roles and their permissions.

2.4 Application Pages

2.4.1 Home — Bond Table

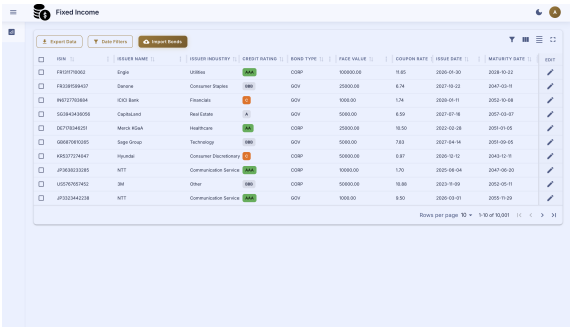


Figure 2.2: Home Bond Table

The home page (/) displays the full bond catalogue in a paginated, server-side-filtered table powered by Material React Table. Each row shows the bond's ISIN, issuer name, country, credit rating, bond type, face value, coupon rate, issue date, and maturity date.

Users can:

- Search by ISIN using the global search box.
- Filter by any column (bond type, credit rating, issuer, date ranges, etc.) using the column filter row.
- Sort by any column by clicking the column header.
- Export the current filtered view to CSV.
- Import bonds from a CSV file (Editors and Admins only).
- Create a new bond via the *Create Bond* button (Editors and Admins only).
- Edit or delete individual bonds inline (Editors and Admins only).
- Select multiple rows and bulk-delete them (Admins only).

2.4.2 Bond Analytics Dashboard

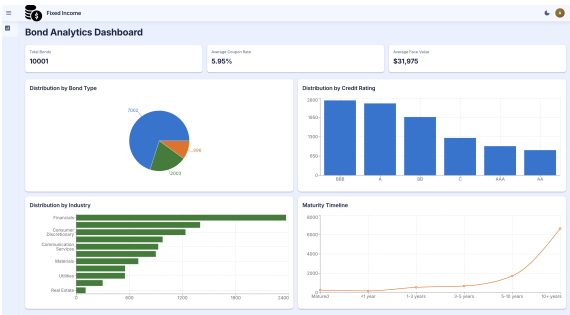


Figure 2.3: Bond Analytics Dashboard

The analytics dashboard (`/dashboard/`) provides a visual overview of the bond catalogue. It calls the `GET /bonds/analytics/` endpoint and renders four Recharts charts:

- **Bond Type Distribution** — pie chart of Government, Corporate, and Municipal bonds.
- **Credit Rating Distribution** — bar chart grouped by issuer credit rating (AAA through C).
- **Industry Distribution** — bar chart of bonds by issuer industry sector.
- **Maturity Timeline** — bar chart bucketing bonds by years to maturity (<1 year, 1–3, 3–5, 5–10, 10+, Matured).

A summary card at the top shows the total number of bonds, the average coupon rate, and the average face value.

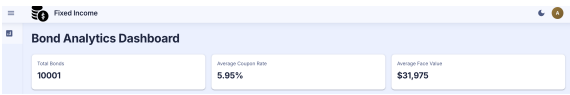


Figure 2.4: Summary cards

2.4.3 Treasury Yield Dashboard

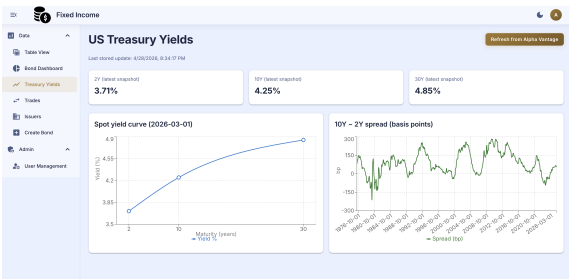


Figure 2.5: Treasury Yield Dashboard

The Treasury dashboard (`/treasury/`) visualises US Treasury constant-maturity yields stored in the database. At the top, three stat cards show the latest snapshot values for the 2-year, 10-year, and 30-year maturities. Below them, two chart cards are displayed side by side:

- **Spot Yield Curve** — a line chart plotting yield against tenor (2, 10, and 30 years) for the most recent date on which all three maturities have an observation. The chart title includes that date so the user knows exactly how current the snapshot is.
- **10Y − 2Y Spread** — a line chart of the historical spread between the 10-year and 2-year yields expressed in basis points, useful for identifying yield-curve inversions.

Editors and Admins can trigger a data sync from the Alpha Vantage API by clicking the *Sync Treasury Yields* button. The sync fetches the full monthly history for all three maturities and upserts the observations into the database.

2.4.4 Issuer Management

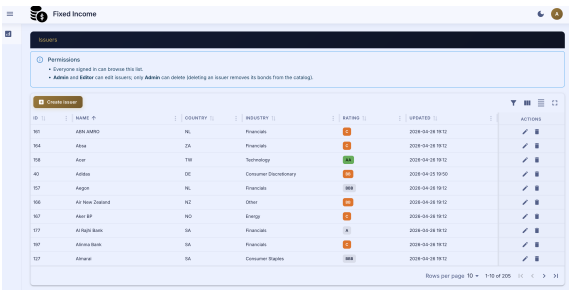


Figure 2.6: Issuer Management

The issuers page (`/issuers/`) lists all bond issuers with their country, industry, and credit rating. Editors and Admins can create new issuers and edit existing ones. Admins can delete issuers (subject to referential integrity — an issuer with associated bonds cannot be deleted).

2.4.5 Transaction Management

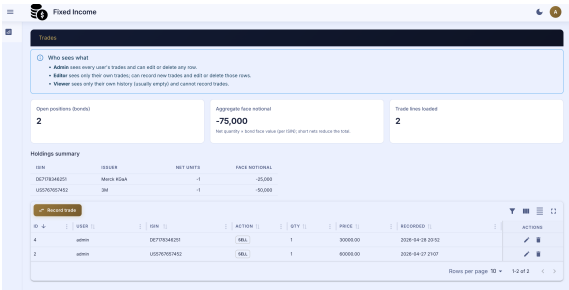


Figure 2.7: Transaction Management

The transactions page (`/transactions/`) records buy and sell events. Each transaction links a user to a bond and captures the quantity and price per bond at the time of the trade. Editors and Admins can create and edit transactions; Admins can delete them.

2.4.6 User Management

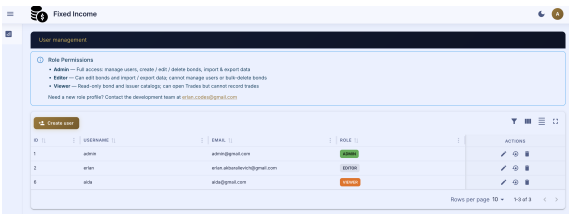


Figure 2.8: User Management

The user management page (`/users/`) is accessible to Admins only. It lists all registered users with their roles. Admins can create new users (with an initial password), edit usernames, emails, and roles, reset passwords, and delete accounts.

2.5 CSV Import and Export

2.5.1 Export

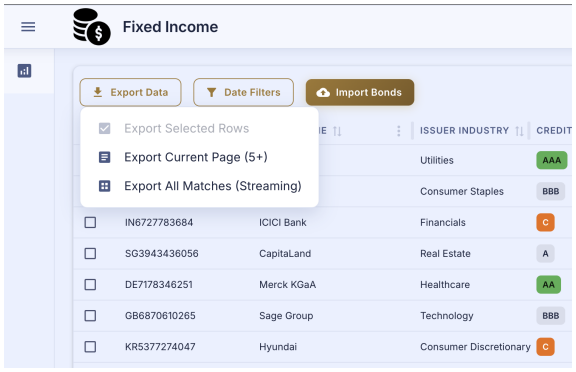
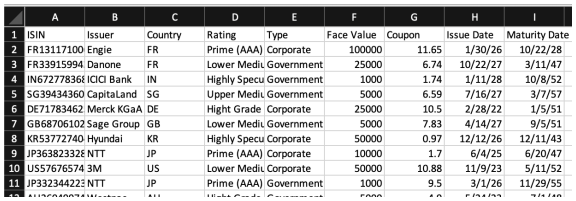


Figure 2.9: Export

Any authenticated user with the Editor or Admin role can export the currently filtered bond list to a CSV file. The export streams the data row-by-row, so it handles large catalogues without loading everything into memory at once.

The exported columns are: ISIN, Issuer, Country, Rating, Type, Face Value, Coupon, Issue Date, Maturity Date.



	A	B	C	D	E	F	G	H	I
1	ISIN	Issuer	Country	Rating	Type	Face Value	Coupon	Issue Date	Maturity Date
2	FR13117100	Engie	FR	Prime (AAA)	Corporate	100000	11.65	1/30/26	10/22/28
3	FR33915994	Danone	FR	Lower Mediu	Government	25000	6.74	10/22/27	3/11/47
4	IN67277836	ICICI Bank	IN	Highly Specu	Government	1000	1.74	1/11/28	10/8/52
5	SG39434360	CapitaLand	SG	Upper Mediu	Government	5000	6.59	7/16/27	3/7/57
6	DE71783462	Merck KGaA	DE	Hight Grade	Corporate	25000	10.5	2/28/22	1/5/51
7	GB68706102	Sage Group	GB	Lower Mediu	Government	5000	7.83	4/14/27	9/5/51
8	KR53772740	Hyundai	KR	Highly Specu	Corporate	50000	0.97	12/12/26	12/11/43
9	JP36382332	NTT	JP	Prime (AAA)	Corporate	10000	1.7	6/4/25	6/20/47
10	US57676574	3M	US	Lower Mediu	Corporate	50000	10.88	11/9/23	5/11/52
11	JP33234422	NTT	JP	Prime (AAA)	Government	1000	9.5	3/1/26	11/29/55

Figure 2.10: Export Result Demo

2.5.2 Import

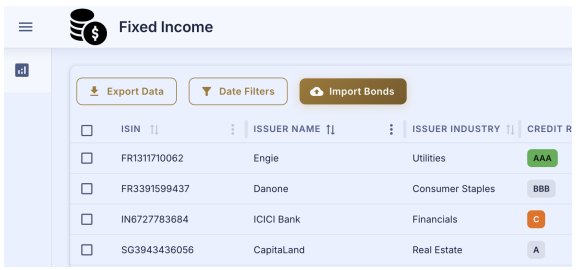


Figure 2.11: Import

The import workflow has two steps:

1. **Preview** — the user uploads a CSV file. The system parses the ISINs and returns a summary: total rows, how many are new, and how many already exist in the database.

2. **Confirm import** — the user confirms and the system processes the file. New bonds (and their issuers, if not already present) are created. Existing bonds are updated with any changed fields. Rows that fail validation are collected into a skipped-rows report that can be downloaded as a separate CSV.

The expected CSV columns are: ISIN, Issuer, Country, Rating, Industry, Type, Face Value, Coupon, Issue Date, Maturity Date.

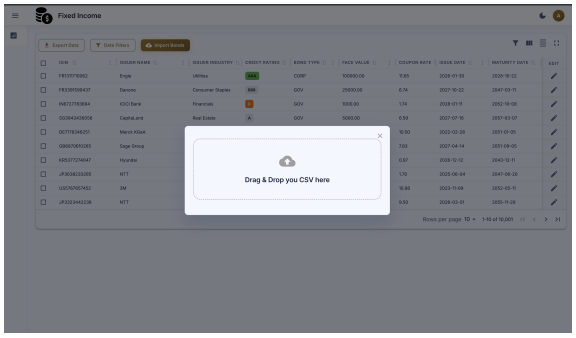


Figure 2.12: Drag Drop

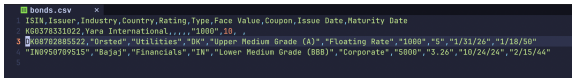


Figure 2.13: Import File

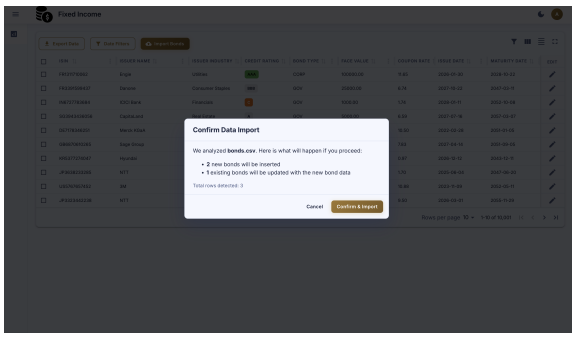


Figure 2.14: Import Preview

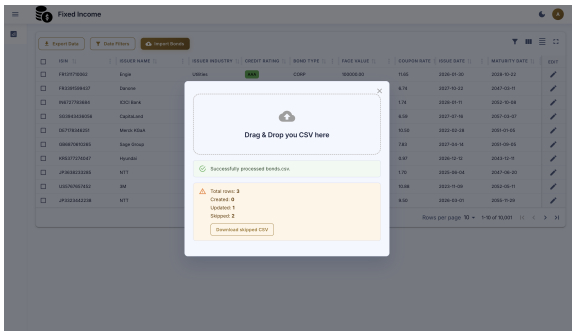


Figure 2.15: Import Result

	A	B	C	D	E	F	G	H	I
1	ISIN	Error							
2	KG03783310	Missing required fields for new bond: Issue Date, Maturity Date							
3	DK08702885	isin: Invalid ISIN format (e.g. US0378331005), Ensure this value has at most 12 characters (it has 13).							
4									

Figure 2.16: Invalid rows

Chapter 3

Developer Documentation

3.1 System Architecture

FixedIncome follows a classic three-tier architecture: a relational database tier, a server-side application tier, and a client-side presentation tier. The three tiers are packaged as independent Docker containers and wired together with Docker Compose.

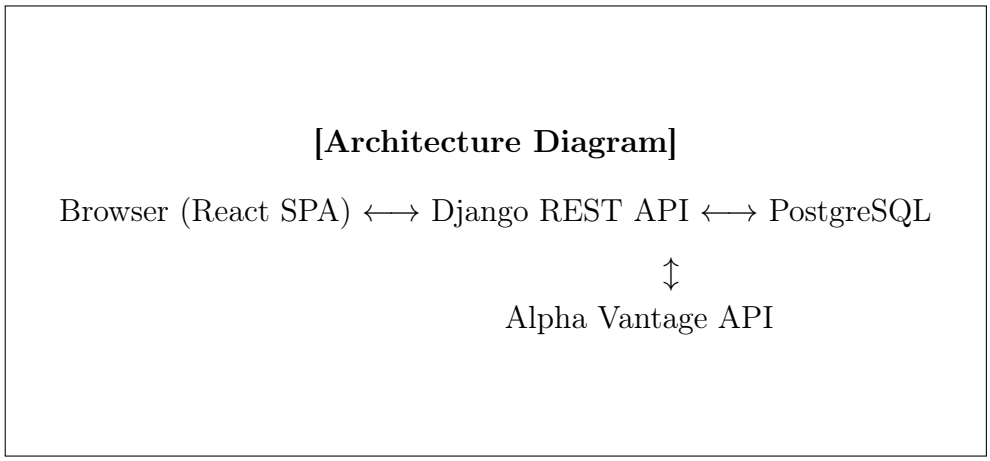


Figure 3.1: High-level system architecture of FixedIncome.

Database (PostgreSQL 16) Stores all persistent data: issuers, bonds, transactions, users, user profiles, and Treasury yield observations. The database runs in a dedicated container with a named Docker volume so data survives container restarts.

Backend (Django 5.2 + Django REST Framework) Implements the business logic, data validation, authentication, and the REST API. It connects

to PostgreSQL via `psycopg2` and to the Alpha Vantage API via the `requests` library.

Frontend (React 19 + Vite) A single-page application that communicates exclusively with the backend API over HTTP. It is served by the Vite development server in the Docker container.

The repository is organised into two top-level service directories and a shared Compose file, as shown below.

```
FixedIncome/
          docker-compose.yml
          README.md
          backend/
                apps/               # Django application code
                pipelines/          # Azure Pipelines CI definitions
                logs/               # Rotating application log files
                htmlcov/            # HTML coverage report (generated)
                Dockerfile
                manage.py
                requirements.txt
                Makefile
                pyrightconfig.json
                db.sqlite3          # SQLite fallback (dev only)
          frontend/
              src/                # React source code
              tests/              # Vitest component tests
              public/             # Static assets
              dist/               # Production build output (generated)
              node_modules/       # npm dependencies (generated)
              Dockerfile
              index.html
              package.json
              vite.config.js
              vitest.config.js
```

Code 3.1: Top-level repository structure.

3.2 Technology Stack

3.2.1 Backend

Library	Version	Purpose
Django	5.2	Web framework, ORM, migrations
Django REST Framework	3.16.1	REST API toolkit, serializers, views
djangorestframework-simplejwt	5.5.1	JWT authentication
django-cors-headers	4.9.0	CORS policy for the SPA
django-filter	25.2	Query-parameter-based filtering
psycopg2	2.9.12	PostgreSQL adapter
python-dotenv	1.2.1	Environment variable loading
requests	2.32.5	HTTP client for Alpha Vantage
coverage	7.13.5	Test coverage measurement
ruff	0.15.11	Linting and formatting

Table 3.1: Backend dependencies.

3.2.2 Frontend

Library	Version	Purpose
React	19.2	UI component library
React Router	7.13	Client-side routing
MUI (Material UI)	7.3	Component library and theming
Material React Table	3.2	Feature-rich data table
Recharts	3.8	SVG-based charting
Axios	1.13	HTTP client with interceptors
React Hook Form	7.72	Form state management
Yup	1.7	Schema-based form validation
Redux Toolkit	(via react-redux)	Global auth state
Vite	7.3	Build tool and dev server
Vitest	4.1	Unit and component testing

Table 3.2: Frontend dependencies.

3.3 Data Model

The database schema is managed entirely through Django migrations. Figure 3.2 gives an overview of the entity relationships.

[Entity-Relationship Diagram]

User (1) — (1) UserProfile
User (1) — (N) Transaction
Issuer (1) — (N) Bond
Bond (1) — (N) Transaction
TreasuryYieldObservation (standalone)

Figure 3.2: Entity-relationship overview of the FixedIncome data model.

3.3.1 Issuer

An `Issuer` represents the entity that issues a bond — a government, a corporation, or a municipality. Key fields are shown in Table 3.3.

Field	Type	Notes
`name`	CharField(255)	Unique; minimum 2 characters
`country`	CharField(100)	ISO country name
`industry`	CharField(100)	Controlled vocabulary (13 sectors)
`credit_rating`	CharField(3)	AAA, AA, A, BBB, BB, C

Table 3.3: Issuer model fields.

3.3.2 Bond

A `Bond` is the central entity. It carries the ISIN identifier, a foreign key to its issuer, financial data, and date information.

Field	Type	Notes
`isin`	CharField(12)	Unique; validated by regex
`issuer`	ForeignKey(Issuer)	CASCADE on delete
`bond_type`	CharField(4)	GOV, CORP, MUNI
`face_value`	DecimalField(15,2)	Principal amount; min 0.01
`coupon_rate`	DecimalField(5,2)	Percentage; 0.00–100.00
`issue_date`	DateField	Must precede maturity date
`maturity_date`	DateField	Cross-field validation in `clean()`

Table 3.4: Bond model fields.

The ISIN format is enforced by a `RegexValidator` requiring two uppercase letters followed by nine alphanumeric characters and a check digit (e.g. `US0378331005`). Cross-field date validation is implemented in the model's `clean()` method.

3.3.3 Transaction

A `Transaction` records a single buy or sell event. It links a `User` to a `Bond` and stores the quantity and price per bond at the time of the trade.

3.3.4 TreasuryYieldObservation

This model stores one data point from the Alpha Vantage `TREASURY_YIELD` endpoint. A unique constraint on `(observation_date, maturity, interval)` prevents duplicate observations and enables idempotent upserts.

Field	Type	Notes
`observation_date`	DateField (indexed)	Date of the yield reading
`maturity`	CharField(10)	2year, 10year, 30year
`interval`	CharField(16)	monthly (only supported value)
`yield_pct`	DecimalField(8,4)	Yield in percent (e.g. 4.2500)
`fetched_at`	DateTimeField	Auto-updated on each upsert

Table 3.5: TreasuryYieldObservation model fields.

3.3.5 UserProfile

Django's built-in `User` model is extended with a one-to-one `UserProfile` that adds a `role` field (ADMIN, EDITOR, VIEWER). The profile is created automatically via a `post_save` signal whenever a new `User` is saved.

3.4 REST API Design

3.4.1 Authentication

The API uses JWT authentication provided by `djangorestframework-simplejwt`. Clients obtain a token pair by `POST`ing credentials to `/api/token/`. The access token (30-minute lifetime) must be included in the `Authorization: Bearer <token>` header of every subsequent request.

The refresh token (7-day lifetime) can be exchanged for a new access token at `/api/token/refresh/`.

3.4.2 Endpoint Overview

Table 3.6 lists the main API endpoints.

Endpoint	Methods	Description
`/api/token/`	POST	Obtain JWT token pair
`/api/token/refresh/`	POST	Refresh access token
`/api/meta/`	GET	API metadata
`/users/`	GET, POST	List / create users
`/users/{id}/`	GET, PATCH, DELETE	User detail
`/users/{id}/reset_password/`	POST	Reset user password
`/users/me/`	GET	Current user profile
`/issuers/`	GET, POST	List / create issuers
`/issuers/{id}/`	GET, PATCH, DELETE	Issuer detail
`/bonds/`	GET, POST	List / create bonds
`/bonds/{id}/`	GET, PATCH, DELETE	Bond detail
`/bonds/analytics/`	GET	Aggregated analytics
`/bonds/bulk_delete/`	DELETE	Delete multiple bonds
`/bonds/export_csv/`	GET	Streaming CSV export
`/bonds/import_preview/`	POST	CSV import preview
`/bonds/import_csv/`	POST	CSV import
`/transactions/`	GET, POST	List / create transactions
`/transactions/{id}/`	GET, PATCH, DELETE	Transaction detail
`/treasury-yields/dashboard/`	GET	Treasury dashboard data
`/treasury-yields/sync/`	POST	Trigger Alpha Vantage sync

Table 3.6: REST API endpoint overview.

3.4.3 Filtering, Searching, and Ordering

The bond list endpoint supports server-side filtering via `django-filter`. Clients can filter on any bond or issuer field using query parameters such as `?bond_type=GOV`, `?coupon_rate__gte=3.0`, or `?issuer__credit_rating=AAA`. Full-text search on the ISIN field is available via `?search=US037`. Results can be ordered by any column with `?ordering=-coupon_rate`.

3.4.4 Pagination

All list endpoints use a custom `StandardResultsSetPagination` class that returns a JSON envelope with `count`, `next`, `previous`, and `results` fields. The default page size is configurable.

3.4.5 Bond Analytics Endpoint

`GET /bonds/analytics/` returns a single JSON object computed by the `build_bond_analytics()` function. The function performs four aggregation queries against the `Bond` and `Issuer` tables:

1. **Summary** — total bond count, average coupon rate, average face value.
2. **By bond type** — count per type (GOV, CORP, MUNI).
3. **By credit rating** — count per issuer credit rating.
4. **By industry** — count per issuer industry sector.
5. **Maturity timeline** — count per maturity bucket (<1 year, 1–3, 3–5, 5–10, 10+, Matured).

```
def build_bond_analytics():
    queryset = Bond.objects.select_related("issuer").all()
    today = date.today()

    by_bond_type = _group_counts(queryset, "bond_type")
    by_credit_rating = _group_counts(
        queryset, "issuer__credit_rating", fallback="Unrated"
    )
    by_industry = _group_counts(
        queryset, "issuer__industry", fallback="Other"
    )

    summary_raw = queryset.aggregate(
        total_bonds=Count("id"),
        average_coupon_rate=Avg("coupon_rate"),
        average_face_value=Avg("face_value"),
    )
    ...
```

Code 3.2: Bond analytics aggregation (excerpt from analytics.py).

3.5 Permission System

Permissions are implemented as DRF `BasePermission` subclasses that inspect the `UserProfile.role` attached to the authenticated user. The four permission classes are:

`IsAdminRole` Grants access only to users with the ADMIN role.

`CanWriteReferenceData` Grants access to ADMIN and EDITOR roles. Used for creating and editing bonds, issuers, and triggering Treasury syncs.

`CanImportExportBonds` Grants access to ADMIN and EDITOR roles. Used for CSV import and export endpoints.

`CanCreateTransaction` Grants access to ADMIN and EDITOR roles. Used for creating and editing transactions.

`IsOwnerOrAdmin` Object-level permission: grants access if the requesting user owns the object or has the ADMIN role.

3.6 Treasury Yield Integration

3.6.1 Alpha Vantage API

The `TREASURY_YIELD` function of the Alpha Vantage API returns a JSON array of `{date, value}` objects for a given maturity and interval. The free tier allows approximately one request per second, so the sync function introduces a configurable delay between requests (default 1.1 seconds).

3.6.2 Sync Flow

The `sync_treasury_yields()` function in `alphavantage_treasury.py` orchestrates the sync:

1. Validate that `ALPHAVANTAGE_API_KEY` is configured.
2. For each maturity (2-year, 10-year, 30-year), call `fetch_treasury_series()` and collect the results in memory.

3. Inside a single database transaction, upsert all observations using `update_or_create()` keyed on `(observation_date, maturity, interval)`.

4. Return a summary dict with total upserted count, per-maturity counts, and the latest observation date for each maturity.

The sync can be triggered via the API (`POST /treasury-yields/sync/`) or via the Django management command `sync_treasury_yields`.

```
with transaction.atomic():
    for maturity, rows in batches:
        for observation_date, yield_pct in rows:
            TreasuryYieldObservation.objects.update_or_create(
                observation_date=observation_date,
                maturity=maturity,
                interval=interval,
                defaults={"yield_pct": yield_pct},
            )
```

Code 3.3: Upsert loop inside sync_treasury_yields().

3.6.3 Dashboard Computation

The `build_treasury_dashboard()` function in `treasury_yield_service.py` reads the stored observations and computes:

- Three time-series arrays (one per maturity) for the historical line charts.
- The most recent yield curve snapshot (the latest date that has all three maturities present).
- The 10Y − 2Y spread series in both percentage points and basis points.

All computation is done in Python from the database; no external API call is made at dashboard load time.

3.7 Frontend Architecture

3.7.1 Project Structure

The React application is bootstrapped with Vite and organised as follows:

- `src/components/` — page-level and reusable UI components.
- `src/components/Forms/` — modal dialogs for create and edit operations.
- `src/components/Navbar/` — navigation bar and drawer menu.
- `src/context/` — React context for authentication state.
- `src/hooks/` — custom React hooks.
- `src/validation/` — Yup validation schemas.
- `src/utils/` — utility functions.
- `src/theme/` — MUI theme configuration.

3.7.2 Authentication Flow

Authentication state is managed by an `AuthProvider` context component. On login, the provider stores the JWT token pair in `localStorage` and exposes the current user and role to all child components. An Axios request interceptor automatically attaches the `Authorization` header to every outgoing request and handles token refresh transparently.

3.7.3 Routing and Access Control

React Router v7 handles client-side routing. Each protected route is wrapped in a `PrivateRouter` component that checks whether the user is authenticated and, optionally, whether their role is in an `allowedRoles` list. Unauthenticated users are redirected to `/login/`; authenticated users without the required role receive a 403 response from the API.

3.7.4 Data Tables

The bond and transaction tables use Material React Table, which provides server-side pagination, filtering, sorting, and row selection out of the box. The component translates its internal state (page index, page size, column filters, sort order) into query parameters and sends them to the backend on every state change.

3.7.5 Charts

All charts are built with Recharts. The analytics dashboard uses `PieChart`, `BarChart`, and `LineChart` components. The Treasury dashboard uses `LineChart` for all four visualisations. Recharts renders SVG natively in the browser, requiring no canvas or WebGL support.

3.8 Logging

The backend uses Python's standard `logging` module configured in `settings.py`. A custom `DefaultUserIdFilter` enriches every log record with the authenticated user's ID and username. Log records are written to both the console and a rotating file (`logs/app.log`, 10 MB per file, five backups). Every API view logs the outcome of each request at the appropriate level (INFO for success, WARNING for permission denials and validation failures).

3.9 Containerisation

The `docker-compose.yml` file defines three services:

`db` PostgreSQL 16 Alpine image. A health check using `pg_isready` ensures the database is ready before the backend starts.

`backend` Built from `backend/Dockerfile`. On startup it runs `python manage.py migrate` and then `runserver`. The source directory is bind-mounted so code changes are reflected immediately without rebuilding the image.

`frontend` Built from `frontend/Dockerfile`. Runs the Vite dev server. The source directory is bind-mounted; `node_modules` is kept inside the container via an anonymous volume to avoid host/container conflicts.

3.10 CI Pipelines

The project uses Azure Pipelines for continuous integration. Two pipeline definitions are maintained under `backend/pipelines/`.

3.10.1 Code Quality Pipeline

`code_quality.yml` is triggered automatically on every pull request targeting the `main` branch. It runs three parallel jobs on an `ubuntu-latest` agent:

BackendTests Sets up Python 3.13, installs dependencies from `requirements.txt`, and runs the Django test suite with `coverage run manage.py test`. The SQLite fallback (`DB_ENV=sqlite`) is used so no external database is needed in CI. A Cobertura XML coverage report is generated and published to Azure DevOps via the `PublishCodeCoverageResults` task.

FrontendTests Sets up Node.js 22, installs dependencies with `npm ci`, and runs the Vitest suite. Results are exported in JUnit XML format and published via the `PublishTestResults` task.

Linter Sets up Python 3.13 and runs `ruff check` against the backend source to enforce code style and catch common errors.

3.10.2 Build and Release Pipeline

`build-release.yml` is triggered manually and delegates to the reusable template `build-release-template.yml`. It accepts two parameters:

- `skipQA` — boolean flag to bypass the code quality stage when a hotfix needs to be released quickly.
- `releaseVersion` — a semantic version string (`Major.Minor.Patch`) used to tag the release commit in Git.

The pipeline runs three sequential stages: **CodeQuality** (same jobs as above, skippable), **BuildRelease** (builds the Python package and releases it to Azure Artifacts), and **TagRelease** (tags the repository commit with the provided version number using a configured Git identity).

Chapter 4

Testing

4.1 Testing Strategy

The project adopts a two-layer testing strategy: unit and integration tests on the backend using Django's built-in test runner, and component tests on the frontend using Vitest with React Testing Library. The goal is to verify the correctness of the API endpoints, the data model constraints, and the key UI interactions.

4.2 Backend Tests

4.2.1 Test Framework

Backend tests are written with Django's `TestCase` class, which wraps each test in a database transaction that is rolled back after the test completes. The `APIClient` from Django REST Framework is used to make HTTP requests to the API views without starting a real server.

Test coverage is measured with the `coverage` library. The coverage report is generated in HTML format and stored in the `htmlcov/` directory.

4.2.2 Running the Tests

```
docker compose exec backend python manage.py test
```

Code 4.1: Running backend tests inside Docker.

To generate a coverage report:

```
docker compose exec backend coverage run manage.py test
docker compose exec backend coverage html
```

Code 4.2: Running backend tests with coverage.

4.2.3 Test Coverage Areas

The backend test suite covers the following areas:

- **Model validation** — ISIN format enforcement, coupon rate bounds, cross-field date validation (maturity must be after issue date).
- **Authentication** — token obtain and refresh endpoints; rejection of requests without a valid token.
- **Role-based access control** — verifying that Viewer-role users cannot create, edit, or delete bonds; that Editor-role users cannot access user management; that Admin-role users have full access.
- **Bond CRUD** — create, retrieve, update, and delete operations via the API, including validation error responses.
- **Issuer CRUD** — same as bonds.
- **Transaction CRUD** — create and retrieve transactions; ownership checks.
- **Analytics endpoint** — verifying the structure and correctness of the aggregated response.
- **CSV export** — verifying the response content type and header row.
- **CSV import** — testing the preview endpoint and the import endpoint with valid and invalid CSV files.
- **Treasury yield sync** — mocking the Alpha Vantage HTTP call and verifying that observations are upserted correctly.

4.3 Frontend Tests

4.3.1 Test Framework

Frontend tests use Vitest as the test runner and React Testing Library for rendering components and simulating user interactions. The `jsdom` environment is configured in `vitest.config.js` to provide a browser-like DOM in Node.js.

4.3.2 Running the Tests

```
docker compose exec frontend npm run test
```

Code 4.3: Running frontend tests inside Docker.

4.3.3 Test Coverage Areas

The frontend test suite includes component tests for:

- **BondAnalyticsDashboard** — verifies that the summary cards and chart containers render when the analytics API returns mock data.
- **TreasuryYieldDashboard** — verifies that the yield series charts and the sync button render correctly.
- **UserManagement** — verifies that the user table renders and that the create-user modal opens on button click.
- **CreateUserModal** — verifies form field rendering and validation feedback.
- **ResetPasswordModal** — verifies that the password reset form submits correctly.
- **IssuerManagement** — verifies that the issuer table renders and that the create-issuer modal is accessible.
- **TransactionManagement** — verifies that the transaction table renders and that the create-transaction modal opens.

4.4 Code Quality

Backend code quality is enforced by `ruff` (linting and formatting). The CI pipeline defined in `pipelines/code_quality.yml` auto runs when a pull request is opened to main branch. The frontend uses ESLint with the `eslint-plugin-react-hooks` and `eslint-plugin-react-refresh` plugins.

Chapter 5

Summary

5.1 Results

This thesis presented the design and implementation of FixedIncome, a full-stack web application for bond portfolio management. The following goals were achieved:

- A normalised relational data model was designed and implemented in Django, covering issuers, bonds, transactions, user profiles, and Treasury yield observations. All domain constraints — ISIN format, coupon rate bounds, date ordering — are enforced at the model layer.
- A RESTful JSON API with 20 endpoints was built using Django REST Framework. The API supports JWT authentication, role-based access control with three roles, server-side filtering and pagination, and streaming CSV export.
- A React single-page application was built with Material UI and Recharts, providing a bond catalogue table, an analytics dashboard with four chart types, a Treasury yield dashboard with historical series and spread analysis, and full CRUD interfaces for all entities.
- The Alpha Vantage `TREASURY_YIELD` API was integrated to fetch and persist monthly US Treasury yields for the 2-year, 10-year, and 30-year maturities. The yield curve snapshot and the 10Y − 2Y spread are derived from the stored data without additional API calls.

- Bulk data operations — CSV import with preview, streaming CSV export, and bulk deletion — were implemented to support practical data management workflows.
- The entire system was containerised with Docker Compose, enabling reproducible one-command deployment.

5.2 Lessons Learned

Several technical insights emerged during development:

- **Streaming responses** are essential for large CSV exports. Django's `StreamingHttpResponse` with a generator avoids loading the entire result set into memory and keeps the server responsive under load.
- **Idempotent upserts** via `update_or_create()` make the Treasury sync safe to run repeatedly without creating duplicate records.
- **Server-side filtering** in the API, rather than fetching all data and filtering on the client, is necessary for tables with thousands of rows.
- **Role-based permissions** are best implemented as reusable permission classes rather than inline `if` statements in each view, which improves consistency and testability.
- **Cross-field validation** belongs in the model's `clean()` method so it is enforced regardless of whether the object is created through the API, the admin interface, or a management command.

5.3 Future Work

Several extensions would make the system more useful in a production context:

- **Yield-to-maturity calculation** — computing the YTM for each bond given a current market price would add significant analytical value.
- **Portfolio P&L tracking** — aggregating transactions per user to show realised and unrealised gains and losses.

- **Real-time price feeds** — integrating a bond pricing API to show current market prices alongside the static face value.
- **Multi-tenancy** — isolating data per organisation so that multiple teams can use the same deployment.
- **Additional yield maturities** — adding 1-month, 3-month, 6-month, 1-year, 5-year, and 7-year Treasury series to produce a complete yield curve.
- **Production deployment** — replacing the Vite dev server with a production build served by Nginx, and adding HTTPS termination.
- **Automated CI/CD** — extending the existing Azure Pipelines configuration to include automated deployment to a cloud environment.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Daniel Varga, for his guidance and constructive feedback throughout the development of this project. I also thank the Faculty of Informatics at Eötvös Loránd University for providing the academic environment that made this work possible.

Bibliography

[1] Charles Schwab. *Bond Investing: Understanding Fixed Income*. YouTube channel. 2024. URL: https://www.youtube.com/@CharlesSchwab (visited on 04/01/2025).

[2] BMO Global Asset Management. *Fixed Income Explained*. YouTube channel. 2024. URL: https://www.youtube.com/@BMOcommunity (visited on 04/01/2025).

[3] The Plain Bagel. *Bonds and Fixed Income Investing*. YouTube channel. 2024. URL: https://www.youtube.com/@ThePlainBagel (visited on 04/01/2025).

[4] ISO. *ISO 6166: Securities and Related Financial Instruments — International Securities Identification Number (ISIN)*. 2021. URL: https://www.iso.org/standard/78502.html (visited on 04/01/2025).

[5] M. Jones, J. Bradley, and N. Sakimura. *JSON Web Token (JWT) — RFC 7519*. 2015. URL: https://datatracker.ietf.org/doc/html/rfc7519 (visited on 04/01/2025).

[6] Alpha Vantage Inc. *Alpha Vantage API Documentation: TREASURY_YIELD*. 2025. URL: https://www.alphavantage.co/documentation/ (visited on 04/01/2025).

[7] Board of Governors of the Federal Reserve System. *Treasury Constant Maturity Rates*. 2025. URL: https://www.federalreserve.gov/releases/h15/ (visited on 04/01/2025).

[8] Django Software Foundation. *Django Documentation*. 2025. URL: https://docs.djangoproject.com/en/5.2/ (visited on 04/01/2025).

[9] Tom Christie and contributors. *Django REST Framework Documentation*. 2025. URL: https://www.django-rest-framework.org/ (visited on 04/01/2025).

[10] Jazzband. *Simple JWT: A JSON Web Token Authentication Plugin for Django REST Framework*. 2025. URL: https://django-rest-framework-simplejwt.readthedocs.io/ (visited on 04/01/2025).

[11] The PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. 2025. URL: https://www.postgresql.org/docs/16/ (visited on 04/01/2025).

[12] Meta Platforms, Inc. *React Documentation*. 2025. URL: https://react.dev/ (visited on 04/01/2025).

[13] MUI SAS. *Material UI Documentation*. 2025. URL: https://mui.com/material-ui/ (visited on 04/01/2025).

[14] Recharts Group. *Recharts: A Composable Charting Library Built on React Components*. 2025. URL: https://recharts.org/ (visited on 04/01/2025).

[15] Evan You and contributors. *Vite Documentation*. 2025. URL: https://vite.dev/ (visited on 04/01/2025).

[16] Vitest contributors. *Vitest Documentation*. 2025. URL: https://vitest.dev/ (visited on 04/01/2025).

[17] Docker Inc. *Docker Documentation*. 2025. URL: https://docs.docker.com/ (visited on 04/01/2025).

List of Figures

2.1 Login 8
2.2 Home Bond Table 9
2.3 Bond Analytics Dashboard 10
2.4 Summary cards 10
2.5 Treasury Yield Dashboard 11
2.6 Issuer Management 11
2.7 Transaction Management 12
2.8 User Management 12
2.9 Export 13
2.10 Export Result Demo 13
2.11 Import 13
2.12 Drag Drop 14
2.13 Import File 14
2.14 Import Preview 14
2.15 Import Result 15
2.16 Invalid rows 15

3.1 High-level system architecture of FixedIncome. 16
3.2 Entity-relationship overview of the FixedIncome data model. 19

List of Tables

2.1 System requirements for running FixedIncome locally. 7
2.2 User roles and their permissions. 8

3.1 Backend dependencies. 18
3.2 Frontend dependencies. 18
3.3 Issuer model fields. 19
3.4 Bond model fields. 19
3.5 TreasuryYieldObservation model fields. 20
3.6 REST API endpoint overview. 21

List of Codes

2.1 Starting the application with Docker Compose. 7
2.2 Stopping the application. 8
2.3 Full teardown including database volume. 8
3.1 Top-level repository structure. 17
3.2 Bond analytics aggregation (excerpt from analytics.py). 22
3.3 Upsert loop inside sync_treasury_yields(). 24
4.1 Running backend tests inside Docker. 28
4.2 Running backend tests with coverage. 29
4.3 Running frontend tests inside Docker. 30